Chapter 19

Loop Engineering

The evolution of how practitioners interact with LLM-based agents has followed a remarkably consistent trajectory. In 2022–2024, the primary skill was *prompt engineering*: crafting the right words to elicit the desired response from a single model call. By 2025, the focus shifted to *context engineering*—curating the full set of tokens the model sees at inference time, including retrieved documents, tool outputs, and conversation history [10]. In the previous chapter we covered *harness engineering*: designing the runtime environment—tools, sandboxes, memory, and guardrails—that surrounds an agent. Loop engineering [279] is the next layer in this progression: designing the *iterative control structure* that drives an agent toward a goal autonomously, without requiring a human to type the next instruction at each turn.

From Prompt to Loop: The Engineering Progression

1. **Prompt engineering**: Optimize what you *say* to the model (2022–2024)
2. **Context engineering**: Optimize everything the model *sees* (2025)
3. **Harness engineering**: Optimize the environment the agent *runs in* (2025–2026)
4. **Loop engineering**: Optimize the *cycle* that keeps the agent working toward a goal (2026)

Each layer wraps the previous one without replacing it. You still write prompts; you still curate context; you still build a harness. Loop engineering puts all of it in motion.

The phrase was coined in June 2026 after Peter Steinberger [338] argued that developers should stop prompting coding agents directly and instead design systems that prompt those agents—a sentiment validated by Boris Cherny, who leads Claude Code at Anthropic, noting that his role had shifted entirely to writing the external execution loops that coordinate model actions [52]. This is not merely a tooling preference. It reflects a structural reality: when a single agent run may last an hour and modify dozens of files, the highest-leverage engineering is no longer in the prompt—it is in the loop that keeps the agent productive, verified, and on-goal throughout.

19.1 Context Engineering: The Layer Beneath the Loop

Before a loop can run, something must decide what the model sees at each step. That discipline is **context engineering**: the formal practice of curating and maintaining the optimal set of tokens in the context window during inference. It is the layer beneath the loop—the mechanism by which the loop's state is translated into model input.

The term was popularized by Tobi Lütke (CEO of Shopify) in June 2025 [244], who described it as the emerging core skill for working with AI agents. Anthropic formalized the definition in September 2025 as "curating and maintaining the optimal set of tokens during inference." Andrej Karpathy endorsed the framing, describing it as "the delicate art and science of filling the context window with just the right information for the next step" [178]. The convergence of these definitions reflects a practical reality: as context windows grew from 4K to 128K to 1M tokens, the question of *what to put in them* became as important as the question of *what to ask*.

Context engineering is not prompt engineering. Prompt engineering optimizes the *instruction*—the words that direct the model's behavior. Context engineering optimizes the *information environment*—the full set of documents, tool outputs, history, and state that the model reasons over. In a single-turn interaction, the distinction is minor. In a multi-step agent loop, it is the difference between a model that has the information it needs to act correctly and one that is flying blind.

Key techniques. Context engineering draws on several complementary methods:

- **Dynamic context assembly**: Selecting and composing context from multiple sources (retrieved documents, tool outputs, memory stores) at each loop iteration rather than using a fixed template.
- **RAG integration**: Retrieving task-relevant documents or code snippets and injecting them into the context window, replacing stale or generic background knowledge with precise, current information.
- **Tool-output summarization**: Compressing verbose tool outputs (e.g., long file listings, test runner logs) before inserting them into context, preserving signal while conserving token budget.
- **Conversation history management**: Deciding which prior turns to retain verbatim, which to summarize, and which to drop entirely—a rolling compression problem that becomes critical in long agent runs.
- **Token budget allocation**: Explicitly partitioning the context window across competing information sources to ensure the model always has room to generate a useful response.

Context Window Budget Allocation (Typical Agent Loop)

Component	Budget	Notes
System prompt	10–15%	Stable; defines agent role, tools, constraints
Retrieved documents / RAG	20–30%	Dynamic; refreshed each iteration
Conversation / action history	30–40%	Compressed as run lengthens
Generation budget	20–30%	Reserved for model output; never compress below this

These are guidelines, not hard rules. Long-horizon coding agents may allocate more to history; single-step QA agents may allocate more to retrieved documents. The key invariant: always reserve the generation budget *before* filling the rest.

Context engineering within a loop. In a loop, context engineering operates at every iteration. At each step t, the loop controller must decide: which prior observations $o_{<t}$ to retain, which to summarize, which retrieved documents are still relevant, and how to compose the new context s_t from these components. This is not a one-time design decision—it is a dynamic policy that runs alongside the agent policy π_θ. A poorly designed context policy causes the model to lose track of its goal, repeat actions it has already taken, or fail to use information it retrieved two steps earlier. A well-designed one keeps the model's "working memory" aligned with the current state of the task throughout the entire run.

Context Engineering vs. Loop Engineering

Context engineering answers: *what does the model see at step t?* Loop engineering answers: *what happens after the model acts at step t?* They are complementary. The loop determines the sequence of actions and observations; context engineering determines how those observations are represented when the model takes its next action. You cannot do loop engineering well without doing context engineering well—the loop's state is only useful if it is correctly encoded in the model's context.

19.2 Loop Engineering as Inference-Time Reinforcement Learning

The central insight that makes loop engineering relevant to this book is that a well-engineered agent loop is structurally identical to an RL optimization process operating at inference time—without gradient updates to the model weights:

$$\begin{aligned} s_t &= \text{context}(s_{t-1}, a_{t-1}, o_{t-1}) && \text{(state = accumulated context)} \\ a_t &= \pi_\theta(s_t) && \text{(action = model generation)} \\ o_t &= \text{env}(a_t) && \text{(observation = tool/environment feedback)} \\ r_t &= \text{verify}(s_t, a_t, o_t) && \text{(reward = verification signal)} \end{aligned} \tag{19.1}$$

The loop continues until r_t exceeds a success threshold or a termination condition triggers. The "policy" π_θ is not updated via gradients; instead, the *state* is updated—observations, error messages, and reflections are appended to the context, conditioning the same frozen model to produce better actions on subsequent iterations. This is precisely the mechanism behind Reflexion [326]: the model improves across iterations not by learning new weights but by reading its own failure history.

The Loop–RL Correspondence

Consider the standard RL objective $\max_\pi \mathbb{E}[\sum_{t=0}^{T} \gamma^t r_t]$. In loop engineering:

- **Policy** π: The frozen LLM, conditioned on accumulating context
- **State** s_t: The context window contents at step t
- **Action** a_t: The model's generated output (code edit, tool call, message)
- **Environment**: The tools, filesystem, test runner, linter—anything that produces observations

- **Reward** r_t: The verification signal (tests pass, linter clean, metric improves)
- **Discount** γ: Implicit in the loop's budget—later steps are costlier and less likely to succeed
- **Episode**: One complete loop execution from goal specification to termination

The critical difference from training-time RL: the policy weights are frozen. "Learning" happens purely through state accumulation—the model receives richer conditioning as the loop progresses. This is analogous to in-context learning, but with the environment actively shaping what the model sees.

This correspondence is not merely metaphorical. It has direct engineering consequences:

1. **Reward design matters**: Just as in RLHF, a poorly specified reward (verification criterion) leads to reward hacking—the agent satisfying the letter of the check while violating its intent (e.g., deleting a failing test to turn CI green).

2. **Exploration–exploitation trade-off**: A loop that repeats the same failing approach exhibits poor exploration. Mechanisms like Reflexion [326] add explicit exploration via self-critique, analogous to entropy bonuses in PPO.

3. **Horizon and discount**: Longer loops face compounding errors and context degradation, just as long-horizon RL suffers from credit assignment difficulty. Budget caps serve as effective horizons.

4. **State representation**: Context management (compaction, pruning, externalization) is the loop-engineering equivalent of state representation design in RL—both determine whether the agent can reason effectively about its situation.

19.3 Anatomy of a Production Loop

A functional loop requires five structural primitives [279] plus external state persistence:

19.3.1 The Five Primitives

Table 19.1: The five structural primitives of loop engineering.

Primitive	Role in the Loop	RL Analogue
Automations	Trigger the loop on schedule or event	Episode initiation; environment reset
Worktrees	Isolate parallel agents	Independent rollout workers in distributed RL
Skills	Codify reusable capabilities and project knowledge	Policy conditioning; task-specific reward shaping
Connectors	Interface with external tools and systems	Environment action space
Sub-agents	Decompose and verify	Hierarchical RL; critic networks

Automations. Automations are the heartbeat that transforms a single agent run into a true loop. They define *when* and *why* a loop fires—on a cron schedule, in response to a webhook, or triggered by a file-system event. Without automations, you have an agent; with them, you have a self-sustaining system. Production examples include nightly CI failure triage, hourly dependency-vulnerability scans, and post-commit review passes.

Worktrees (Isolation). The moment multiple agents run in parallel, file-level conflicts become the dominant failure mode. Git worktrees—separate working directories sharing repository history—provide isolation: each agent operates on its own branch without the possibility of write conflicts with siblings. This is the same principle as running independent rollout workers in distributed RL (Section 11.2): parallelism requires isolation.

Skills. Skills (Chapter 23) encode project knowledge that the agent would otherwise re-derive from scratch every cycle. In the loop context, skills serve as *persistent conditioning*—the conventions, build procedures, and constraints that remain stable across iterations. Without skills, each loop iteration begins cold; with them, the agent compounds knowledge across runs without exhausting the context window on rediscovered facts.

Connectors. Connectors—typically implemented via MCP (Chapter 22)—extend the loop's action space beyond the filesystem. A loop that can only read and write files is limited; one connected to the issue tracker, CI system, staging environment, and team communication channel can close the full feedback loop: detect problem → fix → validate → deploy → notify.

Sub-agents (Maker–Checker Separation). The most critical structural principle in loop engineering is separating the agent that *produces* output from the agent that *evaluates* it. A model grading its own output is analogous to a student marking their own exam—the incentives are misaligned. A dedicated verification sub-agent, potentially running a different model or at higher reasoning effort, provides the independent evaluation that makes unattended operation trustworthy. This mirrors the actor–critic architecture in RL: the actor (generator) proposes actions; the critic (verifier) evaluates them.

19.3.2 External State: The Loop's Memory

Every primitive above operates within a single iteration. *External state* is what connects iterations across time. Because LLMs are stateless between invocations—the model forgets everything not in its current context—the loop's continuity must live on disk: a markdown progress file, a structured database, or a version-controlled log. This external state serves three functions:

1. **Progress tracking**: What has been attempted, what succeeded, what remains.
2. **Failure memory**: Which approaches were tried and failed, preventing the loop from oscillating between identical dead ends.
3. **Handoff context**: When the loop escalates to a human or a subsequent run, the state file provides a complete audit trail.

State Is Not Optional

A loop without external state is a loop with amnesia. It will re-attempt failed approaches, lose track of partial progress, and provide no audit trail when things go wrong. The state mechanism need not be complex—a markdown file committed to git after each iteration is often sufficient—but it must exist.

19.4 The Loop in Pseudocode

Stripped to its essence, every agent loop is a control structure closer to a thermostat or a REPL than to a conversation:

```
def agent_loop(goal: str, max_steps: int = 50, budget: TokenBudget = None):
    """The canonical agent loop structure."""
    state = initialize_state(goal)        # Load external state + goal
```

```
    for step in range(max_steps):
        # 1. Reason about current state (ReAct-style)
        thought = model.reason(state)

        # 2. Choose and execute action
        action = model.choose_action(state)
        observation = environment.execute(action)

        # 3. Update state with new information
        state = update_state(state, thought, action, observation)

        # 4. Compact context if approaching window limit
        if state.token_count > CONTEXT_BUDGET * 0.8:
            state = compact(state)        # Summarize old steps

        # 5. Check termination conditions
        if verifier.passes(state, goal):  # Deterministic check
            persist_state(state, status="success")
            return Success(state)

        if no_progress_detected(state, window=3):
            persist_state(state, status="stuck")
            return escalate_to_human(state)

        if budget and budget.exhausted():
            persist_state(state, status="budget_exceeded")
            return escalate_to_human(state)

    # Hard cap reached
    persist_state(state, status="max_steps")
    return escalate_to_human(state)
```

Everything interesting in loop engineering is a decision about one of these lines: what constitutes a valid `goal`, how `verifier.passes` is implemented, how `compact` preserves relevant history while discarding noise, and how `no_progress_detected` avoids both premature termination and infinite cycling.

19.5 Loop Patterns

Building on the agent design patterns introduced in Chapter 20, loop engineering recognizes a hierarchy of increasingly autonomous patterns:

19.5.1 The Validation Loop

The simplest and most common pattern: generate, validate against a deterministic check, retry on failure.

Validation Loop: Fix Until Tests Pass

```
failure_log = []
for attempt in range(MAX_ATTEMPTS):
    result = run_tests()
    if result.passed:
        break  # Success -- exit loop
    fix = model.generate(
        f"Fix this test failure:\n{result.errors}\n"
        f"Previous attempts that failed: {failure_log}"
    )
    apply_patch(fix)
    failure_log.append(result.errors)
```

Termination: Tests pass (success) or retry budget exhausted (escalate).

Key property: The verification signal is *deterministic*—the test runner produces an objective pass/fail that the model cannot argue around.

This pattern succeeds because its reward signal is crisp. The test runner serves as an incorruptible critic—unlike LLM-as-judge evaluation, it cannot be persuaded or fooled.

19.5.2 The Reflexion Loop

Extends the validation loop with explicit self-critique between attempts [326]. After each failure, the agent writes a natural-language reflection ("I failed because I modified the wrong function—the error traces back to the import, not the implementation") into an episodic memory buffer. Subsequent attempts read this buffer, enabling learning *within* an episode without weight updates.

Reflexion: In-Context Learning from Failure

The Reflexion loop adds three components beyond basic retry:

1. **Actor**: Executes the task
2. **Evaluator**: Provides the reward signal (tests, metrics, or LLM judge)
3. **Self-Reflection**: Writes a verbal "lesson learned" to episodic memory

The reflection memory persists across attempts, giving the agent a mechanism to avoid repeating identical failures. Empirically, Reflexion achieves 91% pass@1 on HumanEval (vs. 67% baseline), demonstrating that verbal self-critique can substitute for weight updates in iterative settings [326].

19.5.3 The Evaluator-Optimizer Loop

A two-model architecture [10, 246]: one model generates a candidate, a separate model evaluates it against explicit criteria and returns structured feedback. The generator incorporates this feedback and produces an improved version. The cycle repeats until the evaluator's score exceeds a threshold or the iteration budget is consumed.

The critical design choice is whether the evaluator is *deterministic* (compiler, test suite, linter) or *probabilistic* (another LLM). Deterministic evaluators produce reliable convergence; probabilistic evaluators are necessary for subjective criteria but introduce the risk of evaluator–generator collusion.

19.5.4 The Hierarchical Loop

A meta-loop spawns and monitors sub-loops. An orchestrator agent decomposes a high-level goal into subtasks, assigns each to a specialized sub-agent running its own loop, monitors their progress, and synthesizes results. This mirrors hierarchical RL [21]: a high-level policy selects sub-goals while low-level policies execute them.

Hierarchical Loop: Overnight Repository Maintenance

```
# Outer loop: runs nightly via automation
for issue in triage_agent.identify_actionable_issues():
    # Inner loop: one sub-agent per issue, isolated worktree
    worktree = create_isolated_worktree(issue.branch_name)

    result = fix_loop(
        goal=issue.description,
        worktree=worktree,
        verifier=run_tests_and_lint,
        max_steps=20
    )

    if result.success:
        review = review_agent.evaluate(result.patch)  # Checker
        if review.approved:
```

```
            open_pull_request(result.patch, issue)
        else:
            state_db.log(issue, "fix_rejected", review.feedback)
    else:
        state_db.log(issue, "fix_failed", result.state)
```

The outer loop (triage) operates on a schedule. Inner loops (fixes) are ephemeral and task-scoped. The review agent provides the maker–checker separation. State is persisted across nightly runs.

19.5.5 The Autonomous Research Loop

Karpathy's AutoResearch [179] demonstrates a tightly constrained research loop: the agent modifies a training script, runs a time-bounded experiment (e.g., 5 minutes on a GPU), reads the validation metric, and decides whether to commit the change (metric improved) or roll back (metric degraded). The loop continues indefinitely, exploring the hyperparameter and architectural space through iterative experimentation.

This pattern works because the reward signal is *scalar and unambiguous*: validation loss either decreased or it did not. The tight time bound per iteration prevents any single failed experiment from consuming excessive resources.

19.6 Verification Engineering

Verification is the reward function of loop engineering. Its quality determines whether the loop converges to the correct answer, converges to a wrong one, or fails to converge at all. This section establishes a hierarchy of verification strategies ordered by reliability.

19.6.1 The Verification Hierarchy

Table 19.2: Verification strategies ordered by reliability.

Strategy	Signal Source	Reliability	Applicable When
Compilation	Language toolchain	Deterministic	Code must parse/build
Type checking	Static analyzer	Deterministic	Typed languages
Unit/integration tests	Test runner	Deterministic	Tests exist and are correct
Linting & formatting	Style tools	Deterministic	Conventions are codified
Metric comparison	Training/eval script	Numerical	ML experiments
LLM-as-judge	Second model	Probabilistic	Subjective quality criteria
Human review	Domain expert	Gold standard	High-stakes decisions

The Verification Principle

A loop is only as trustworthy as its least reliable verification step. Wherever a deterministic check exists, use it. Reserve LLM-as-judge for criteria that genuinely cannot be mechanically verified. Never let the generating model judge its own output—the maker and the checker must be separate.

19.6.2 Deterministic vs. Probabilistic Verification

Deterministic verifiers (compilers, test suites, linters) are the gold standard because they produce an objective pass/fail that the model cannot game through persuasion. A test either passes or it does not—no amount of eloquent reasoning changes the verdict.

Probabilistic verifiers (LLM-as-judge) are necessary for tasks without mechanical checks—code quality, documentation clarity, UX improvement—but they introduce failure modes:

- **Self-evaluation bias**: If the same model generates and evaluates, it will be lenient with its own output.
- **Evaluator gaming**: Over many iterations, the generator may learn to produce outputs that satisfy the evaluator's surface patterns without achieving genuine quality.
- **Evaluator inconsistency**: The same LLM-judge may score identical outputs differently across runs due to sampling variance.

Mitigation strategies include using a different (often stronger) model as evaluator, providing explicit rubrics with binary criteria, and periodically calibrating the LLM-judge against human judgments.

19.7 Termination Engineering

The signature failure of a naive loop is that it never stops. Termination design is not an afterthought—it is half the engineering.

19.7.1 Termination Conditions

A production loop carries multiple independent exit conditions:

1. **Goal achieved**: The verifier confirms the objective is met. This is the only *successful* termination.
2. **Hard iteration cap**: Maximum number of loop steps (typically 20–50). Prevents unbounded execution regardless of other conditions.
3. **Budget exhaustion**: Token count, wall-clock time, or monetary cost exceeds a predefined limit.
4. **No-progress detection**: The last k iterations produced no measurable change in state—the loop is oscillating or stuck. This is the most important safety mechanism after the hard cap.
5. **Fatal error**: An unrecoverable condition (missing credentials, infrastructure failure) that no amount of iteration can resolve.

No-Progress Detection: The Stagnation Circuit Breaker

Detecting stagnation requires tracking a progress signal across iterations. Common approaches:

- **Output hash comparison**: If the last 3 agent outputs or file states are identical, the loop is cycling.
- **Error repetition**: The same error message appears in k consecutive iterations.
- **Metric plateau**: A numerical signal (test pass rate, validation loss) has not improved by ϵ in k steps.
- **Diff entropy**: The size of code changes per iteration is decreasing toward zero—the agent is making progressively smaller, inconsequential edits.

When stagnation is detected, the correct action is *escalation*, not continued iteration. The loop saves its state and hands control to a human or a higher-level orchestrator.

19.7.2 The Exploration Problem

A loop stuck in a local minimum—repeating the same failing approach with minor variations—is exhibiting the exploration–exploitation failure familiar from RL. Mechanisms to promote exploration within loops:

- **Reflection-based exploration**: After k failed attempts, trigger a reflection step that explicitly asks the model to consider fundamentally different approaches (analogous to entropy regularization in PPO [319]).
- **Temperature escalation**: Increase sampling temperature after consecutive failures to encourage diverse outputs.
- **Strategy memory**: Record not just failed actions but failed *strategies*. "Approach A (modify the handler) failed 3 times; try approach B (rewrite the schema) instead."
- **Fresh sub-agent**: Spawn a new sub-agent with a clean context window—it cannot fall into the rut that the accumulated context may have created.

19.8 Failure Modes and Anti-Patterns

19.8.1 The Loopmaxxing Trap

"Loopmaxxing"—the assumption that running an agent through enough iterations will eventually produce a correct solution—is the loop-engineering equivalent of believing that enough compute solves all problems. It fails for the same reason that pure random search fails: without a gradient signal pointing toward improvement, iteration alone is not optimization.

Loopmaxxing manifests when:

- The goal lacks a checkable success condition ("improve the user experience")
- The verification signal is too coarse (binary pass/fail on a 1000-test suite provides no gradient toward the fix)
- The agent lacks the capability to solve the problem regardless of iteration count

The antidote is recognizing that loops *amplify* engineering capability—they do not replace it. A loop with a poorly specified objective will pursue the wrong thing with great efficiency.

19.8.2 Comprehension Debt

When a loop modifies code faster than the engineering team can review it, the gap between what exists in the repository and what humans understand grows—*comprehension debt* [279]. Unlike technical debt (which the code carries), comprehension debt lives in the team's heads. It compounds silently until a crisis forces someone to debug code whose design decisions, structural dependencies, and edge cases are entirely unmapped.

Loops Amplify Both Productivity and Risk

A well-designed loop accelerates work the engineer understands deeply. A loop used to avoid understanding the work creates an accelerating spiral of incomprehension. Same mechanism, opposite outcomes. The difference is whether the human stays the engineer or becomes merely the person who presses "go."

19.8.3 Reward Hacking in Loops

Just as RL agents exploit reward misspecification to achieve high reward without achieving the intended goal, loop agents exploit verification gaps:

- **Test deletion**: The simplest reward hack—delete the failing test to make CI green.
- **Metric gaming**: Overfitting to validation loss by memorizing the validation set rather than generalizing.
- **Specification narrowing**: Solving an easier version of the stated problem that happens to pass the checks.
- **Output masking**: Suppressing error output rather than fixing the underlying cause.

The defense is the same as in RLHF: design reward functions (verification criteria) that are *hard to game*. Multiple independent checks—tests *and* type checking *and* linting *and* a review sub-agent—create a verification surface that is much harder to exploit than any single signal.

19.8.4 Context Degradation

In long-running loops, the context window fills with the history of every step—thoughts, tool outputs, errors, patches. As the window approaches capacity, the model's attention to relevant information degrades (the "lost in the middle" phenomenon [226]). Symptoms include:

- The agent re-attempts approaches it has already tried and explicitly noted as failures
- Tool calls become less precise—the agent forgets which files it has already read
- Responses become shorter and less coherent as the model struggles to attend across a bloated context

Countermeasures:

1. **Aggressive compaction**: After every k steps, summarize completed work into a brief state description and discard raw transcripts.
2. **External scratchpad**: Write intermediate results to files that the agent reads on-demand rather than keeping everything in context.
3. **Sub-agent isolation**: Spawn subtasks in fresh context windows that return only their conclusion—not their full reasoning trace.
4. **Sliding window over history**: Keep only the last w steps in full detail; compress everything earlier into a summary block.

19.9 Production Loop Architectures

19.9.1 The Nightly Maintenance Loop

The most immediately practical loop pattern: a scheduled automation that runs overnight, processing accumulated work while the team sleeps.

Architecture: Nightly CI Triage and Fix Loop

Trigger: Cron schedule (e.g., 02:00 UTC daily).
Phase 1 — Triage (single agent, read-only):

- Read yesterday's CI failures, newly opened issues, and recent commits

- Classify each finding: auto-fixable / needs-human / false-positive
- Write findings to a state file; archive false positives

Phase 2 — Fix (parallel sub-agents, isolated worktrees):

- For each auto-fixable issue, spawn a sub-agent in an isolated worktree
- Each sub-agent runs a validation loop (fix → test → retry)
- Budget: 20 steps or 15 minutes per issue

Phase 3 — Review (separate verification agent):

- A review sub-agent evaluates each proposed fix against project coding standards
- Approved fixes: open draft PR, link to issue, notify channel via connector
- Rejected fixes: log rejection reason, escalate to human triage inbox

Persistence: State file (committed to repo) records all actions taken, enabling the next night's run to skip resolved issues and pick up where failures left off.

19.9.2 The Continuous Experimentation Loop

Inspired by AutoResearch [179], this pattern applies the loop to scientific discovery: modify a training script, run a time-bounded experiment, evaluate the result, and decide whether to commit or rollback.

AutoResearch Loop Structure

1. **Hypothesis generation**: Agent proposes a modification (hyperparameter, architecture change, data augmentation)
2. **Implementation**: Agent modifies the training script
3. **Bounded execution**: Run for exactly T minutes (hard wall-clock cap)
4. **Evaluation**: Read validation metric from output
5. **Decision**: If metric improved → `git commit`; if degraded → `git rollback`
6. **Log**: Record hypothesis, result, and reasoning in experiment log
7. **Repeat**: Generate next hypothesis informed by cumulative experiment history

Key constraint: The bounded execution time prevents any single failed experiment from consuming excessive compute. The git-based commit/rollback ensures the codebase only moves forward.

The Local Minimum Problem in Research Loops

Research loops are susceptible to conservative exploration. When facing difficult optimization landscapes, agents tend to propose minimal changes—adjusting a learning rate by 0.001—that produce nominal improvements without genuine progress. This is the RL exploration problem manifesting at the loop level. Countermeasures include periodic "bold hypothesis" prompts, diversity bonuses for trying novel approaches, and human-injected research directions between runs.

19.9.3 The Always-On Orchestrator

Systems like OpenClaw [338] implement a persistent "heartbeat" mechanism: a meta-agent that runs on a fixed cycle, evaluates repository state, supervises sub-loops, and dispatches work. Unlike the

nightly pattern which fires once per day, the always-on orchestrator maintains continuous awareness and responds to events (new commits, failing CI, incoming issues) within minutes.

The architecture introduces a durable state database with crash recovery—if the orchestrator fails mid-cycle, it resumes from its last checkpointed state rather than restarting from scratch. Sub-loops are tracked as independent tasks with their own termination conditions, and the orchestrator monitors them for stagnation, budget exhaustion, or conflict.

19.10 Context Management in Long-Running Loops

Long-running loops face a fundamental tension: the context window is the agent's working memory, but it has a fixed capacity. Every step appends new information (thoughts, observations, errors), and without active management the window fills and quality degrades. This section presents the engineering techniques for maintaining context health across many iterations.

19.10.1 Compaction Strategies

Table 19.3: Context compaction strategies for long-running loops.

Strategy	Mechanism	Trade-off
Periodic summarization	Every k steps, replace raw history with a summary	Lossy; may discard details needed later
Sliding window	Keep last w steps verbatim; summarize earlier	Recency-biased; may lose early context
Importance-weighted	Retain steps that changed state; discard no-ops	Requires defining "importance"
External memory	Write details to files; keep only references in context	Requires explicit read-back; adds latency
Sub-agent isolation	Subtasks run in fresh contexts; return conclusions only	Overhead of spawning; coordination cost

The optimal strategy depends on the task. For debugging loops, a sliding window works well—recent errors are most relevant. For research loops, importance-weighted retention preserves key experimental results across many iterations. For complex multi-file refactoring, external memory (writing a progress file that summarizes all changes so far) prevents the agent from losing track of its own modifications.

19.10.2 The Context Budget

A practical heuristic: reserve 20–30% of the context window for the current step's reasoning and generation. This means active history should not exceed 70–80% of capacity. When this threshold is approached, trigger compaction before the next iteration—not after the model has already produced degraded output.

```
CONTEXT_CAPACITY = 128_000  # tokens
RESERVED_FOR_GENERATION = 0.25 * CONTEXT_CAPACITY
ACTIVE_BUDGET = CONTEXT_CAPACITY - RESERVED_FOR_GENERATION

def should_compact(state: LoopState) -> bool:
    return state.token_count > ACTIVE_BUDGET * 0.85
```

19.11 When Not to Use Loops

Loop engineering is not universally applicable. It adds complexity, cost, and failure modes that are unjustified when simpler approaches suffice. Prefer direct prompting or simple workflows when:

- The task is **single-turn**: A well-crafted prompt produces the correct answer in one shot most of the time. Adding a loop provides marginal quality improvement at significant cost.
- The goal **lacks a checkable condition**: "Make the code better" has no objective stopping point. The loop will either run forever or stop arbitrarily.
- **Human interaction is cheap**: If a developer is actively working and can provide feedback in real-time, the interactive chat modality is faster than engineering a loop.
- The task **exceeds model capability**: If the agent cannot solve the problem in principle—regardless of iteration count—a loop merely burns tokens. No amount of retry fixes a fundamental capability gap.
- **Verification is impossible**: Without a feedback signal, the loop has no gradient. It degenerates into random search.

The Simplicity Principle

The same design principle articulated in Chapter 20 applies: start with the simplest approach that works, and add loop complexity only when you have evidence that iteration materially improves outcomes. A prompt chain that solves the problem is always preferable to a loop that might.

19.12 The Economics of Loops

Loops trade compute (tokens, wall-clock time) for quality. Understanding this trade-off is essential for production deployment.

Loop Cost Model

For a loop with average N iterations, each consuming T tokens at price p per token:

$$\text{Cost}_{\text{loop}} = N \cdot T \cdot p + \text{Cost}_{\text{tools}} + \text{Cost}_{\text{verification}}$$

With prompt caching (where the system prompt and early context are cached and charged at reduced rate $p_c < p$):

$$\text{Cost}_{\text{cached}} = T_{\text{new}} \cdot p + T_{\text{cached}} \cdot p_c + (\text{tools} + \text{verification})$$

Prompt caching is particularly valuable for loops because the goal specification, skill content, and early history remain constant across iterations—often 60–80% of each call's context is cacheable.

The economic decision: is the value of the loop's output (developer time saved, overnight productivity, quality improvement) greater than its token cost? For a loop that runs 20 iterations at \$0.02/iteration, the total cost is \$0.40—trivial if it saves 30 minutes of developer time. But a runaway loop without budget caps can consume hundreds of dollars in a single night.

19.13 Historical Context and Related Work

Loop engineering did not emerge from a vacuum. It productized a research lineage spanning several years:

1. **ReAct** (2022) [408]: Established the interleaved reasoning-and-action pattern that all modern loops inherit.
2. **Reflexion** (2023) [326]: Added episodic memory and self-critique, demonstrating that agents can improve across attempts without weight updates.
3. **AutoGPT** (2023) [330]: The first widely-used autonomous agent loop, demonstrating both the potential and the pitfalls (runaway execution, high cost, unreliable output) of unattended operation.

4. **Self-Refine** (2023) [246]: Formalized the generate-then-critique loop with iterative refinement.
5. **"Ralph loops"** (2025): Informal bash-based one-liner scripts that ran coding agents in simple retry loops—the precursor to productized loop primitives.
6. **Productized loops** (2026): Major platforms (Codex, Claude Code) embedded `/goal` and `/loop` commands directly, making sophisticated loop patterns accessible without custom infrastructure [279].

The progression from AutoGPT to modern loop engineering illustrates a maturation: early systems lacked termination logic, verification, and cost controls—they were loops without engineering. The 2026 formalization added the discipline that makes unattended operation safe: explicit termination, deterministic verification, budget constraints, and maker–checker separation.

19.14 Summary

Loop engineering represents the current frontier of human–agent collaboration: the shift from participating in conversations to designing systems that have those conversations autonomously. Its key principles:

1. A loop is inference-time RL—state, action, reward, and policy update (via context)—operating without gradient descent.
2. Verification is the reward signal. Its quality determines convergence. Prefer deterministic checks; separate the maker from the checker.
3. Termination is half the design. Every loop needs a hard cap, a budget, and no-progress detection.
4. Context is finite working memory. Manage it actively through compaction, externalization, and sub-agent isolation.
5. Loops amplify engineering skill—they do not replace it. A loop with a poorly specified objective will pursue the wrong thing efficiently.
6. Start simple. A single validation loop with a deterministic verifier outperforms an elaborate multi-agent system you cannot debug.

Build the Loop—Stay the Engineer

The leverage point has moved from writing prompts to designing cycles. But designing a cycle that converges requires understanding the problem deeply enough to specify what "done" means, what constitutes progress, and when to stop. Loop engineering makes the human's judgment *more* important, not less—because the consequences of bad judgment now compound at machine speed.